



Milestone 3 — Test & Bug-Fix Report

Implementation & Prelaunch

Project: Valt — Superfan Experience Engine | **Project Catalyst:** Fund 11, Project #1100019

Milestone: 3 — Implementation & Prelaunch | **Live portal:** <https://www.valt.digital> | **Network:** Cardano pre-production testnet

Prepared: 16 July 2026 | **Plugin version:** valt-platform 2.0.2

1. Purpose & Criterion Mapping

This report is the evidence for the Milestone 3 Outputs *"Conduct testing of the features and functionalities"* and *"Identify and resolve any bugs or issues"*, and for the Acceptance Criterion *"Completion of testing and bug fixing"*. Security work is reported separately in the companion **M3 Security Audit Report**; user feedback and the revised roadmap in **M3 User Feedback & Roadmap**.

#	Acceptance criterion	Status	Where
1	Completion of testing and bug fixing	Met	Automated suite \$4 (15 tests, 25 assertions, all passing); bugs \$5 (6 identified, 6 resolved)
2	Passes all vulnerability checks conducted through automated cybersecurity tools	Met	\$6 and the companion Security Audit Report
3	Passes all manual security assessments documented by the team	Met	Companion Security Audit Report
4	User feedback collected and analysed for further improvements	See companion	M3 User Feedback & Roadmap

2. Headline Finding — the published plugin could not be installed

The single most important result of this milestone is that **testing found a real, severe defect in the artifact Project Catalyst grades**. The open-source build published at github.com/Awen-online/valt — the repository cited as the "GitHub repo updated with code" evidence — contained a **plugin that could not be activated at all**. Any reviewer who cloned the repository and activated `valt-platform` would have received a PHP fatal error immediately, before any feature could be exercised.

We are reporting this prominently rather than quietly folding the fix into a change list. A testing milestone that discovers nothing is not evidence that testing happened. This defect was present during the Milestone 2 review and was not detected, which tells us something useful about the difference between *publishing* code and *verifying* the published code runs. The whole of \$4 exists to make sure that gap cannot reopen.

Root cause

`valt-platform.php` loaded its optional modules behind a `file_exists()` guard, but two files were required unconditionally:

```
require VALT_PLATFORM_PATH . 'includes/admin-docs.php'; // line 25
require VALT_PLATFORM_PATH . 'seed-data.php';           // line 34
```

Neither file ships in the curated public build (both are intentionally excluded as non-milestone tooling). PHP therefore aborted at include time:

```
PHP Fatal error: Uncaught Error: Failed opening required '../includes/admin-docs.php'
in ../valt-platform/valt-platform.php:25
```

Verification — before and after

A load harness (`tests/load-plugin.php`) stubs the WordPress API and loads the published build in a clean process. Run against the repository as it stood, and against the fixed build:

BEFORE (public repo at previous HEAD)	AFTER (this milestone)
VERDICT=FATAL	VERDICT=LOADED
Failed opening required	VERSION=2.0.2
'includes/admin-docs.php'	HOOKS=61
at valt-platform.php:25	SHORTCODES=24

Fix. The two requires were moved into the same `file_exists()` pattern the file already used for every other optional module, preserving load order so that `admin-docs.php` still registers its parent admin menu before `admin-nft-monitor.php` attaches a submenu to it. Production was never affected — `valt.digital` runs the full build, where both files are present.

3. Test Infrastructure

Before this milestone the project had **no automated tests, no continuous integration, and no bug tracker**. All verification to date had been manual screenshot capture and on-chain confirmation. That was sufficient to demonstrate Milestone 2's features existed; it could not detect that the published build was broken, because nobody ever loaded it.

Milestone 3 introduces a test suite deliberately shaped around the defects that actually occurred, rather than around coverage metrics. Every test below corresponds to a bug found in this milestone.

Component	Purpose
tests/Analyzer.php	Static analysis over the build using PHP's own tokenizer (<code>token_get_all</code>) rather than regular expressions, so comments and SQL string literals cannot be mistaken for code
tests/wp-stubs.php	Minimal WordPress API surface — enough to load the plugin without a WordPress install
tests/load-plugin.php	Loads a build in a clean subprocess and reports a machine-readable verdict, so a fatal is observable instead of killing the runner
tests/PluginLoadTest.php	The plugin loads; every unconditional require resolves; graded shortcodes register
tests/StaticAnalysisTest.php	Syntax; version consistency; no unguarded calls to undefined functions; feature-flag defaults; component coupling
tests/SecurityTest.php	REST permission callbacks; AJAX nonces; CSRF on destructive admin actions; dangerous constructs; committed secrets
.github/workflows/ci.yml	Runs lint + suite on PHP 8.0/8.1/8.2/8.3, plus a semgrep security scan, on every push and pull request

4. Test Results

Executed against the published open-source build (`code/` : 59 PHP files — 16 in `valt-platform` , 43 in `valt-theme`), on PHP 8.2.29.

```
PHPUnit 10.5.64 by Sebastian Bergmann and contributors.
Runtime: PHP 8.2.29

.....                                     15 / 15 (100%)

OK (15 tests, 25 assertions)
```

The suite is proven, not assumed

A suite that passes proves nothing on its own. Re-running it against the code as it stood *before* this milestone's fixes produces exactly four failures, each one a real defect:

```
1) PluginLoadTest::test_plugin_loads_without_fatal
   VERDICT=FATAL
2) PluginLoadTest::test_every_unconditional_require_target_exists
   valt-platform.php:25 requires missing includes/admin-docs.php
   valt-platform.php:34 requires missing seed-data.php
3) PluginLoadTest::test_plugin_registers_its_shortcodes
4) StaticAnalysisTest::test_plugin_version_is_consistent
   Plugin header Version (2.0.0) and VALT_PLATFORM_VERSION (2.0.1) must match

Tests: 15, Assertions: 20, Failures: 4.
```

Four failures before the fixes; zero after. This is the property that matters: the suite fails when the software is broken.

Syntax verification

`php -l` across every PHP file in all three working trees — the published build, the full private build, and the local development sandbox — reports **0 failures** in 62 files.

5. Bugs Identified and Resolved

Six defects were identified during this milestone. All six are resolved. Severity reflects impact on a user of the published artifact.

#	Severity	Defect	Resolution
1	Critical	Published plugin fatal on load. <code>valt-platform.php</code> unconditionally required <code>includes/admin-docs.php</code> and <code>seed-data.php</code> , neither of which ships in the curated public build.	Both moved behind the existing <code>file_exists()</code> guard, preserving admin-menu load order. Verified by load harness: FATAL → LOADED.
2	Critical	Undefined function fatal. <code>valt_resolve_artist_id()</code> was called unguarded from <code>[valt_song_card]</code> and <code>[valt_song_grid]</code> — the home-page grid — but the function itself was missing from the published build. The fix existed in the private repository and had never been propagated.	Function carried forward into <code>helpers.php</code> in every tree. All three trees are now byte-identical for the shared modules.
3	Medium	Latent fatals via REST. Three unguarded calls into the excluded Stripe module — <code>valt_create_checkout_session()</code> , <code>valt_handle_stripe_webhook()</code> , <code>valt_sync_song_to_stripe()</code> — reachable in the public build because the <code>stripe</code> feature flag defaults to enabled.	Guarded with <code>function_exists()</code> ; the REST routes now return HTTP 501 with a clear message instead of fatalling.
4	Medium	CSRF on destructive admin actions. Both seeders performed state-changing writes on a plain <code>GET ?confirm=yes</code> , protected by a capability check but no nonce. The page seeder removes Elementor data from replaced pages.	<code>wp_nonce_url()</code> on the trigger and <code>check_admin_referer()</code> in the handler, matching the pattern already used correctly by the NFT Monitor.
5	Low	Stored XSS in the artist dashboard. <code>\$song_album</code> and <code>\$song_duration</code> were echoed unescaped. Both derive from post meta, which can be set through Pods or wp-admin, bypassing the sanitising AJAX path.	<code>esc_html()</code> at output. The fallback value was the HTML entity <code>&mdash;</code> — the reason escaping had been omitted — so it was changed to a literal em-dash character.
6	Low	Version mismatch. Plugin header declared <code>2.0.0</code> while <code>VALT_PLATFORM_VERSION</code> declared <code>2.0.1</code> . WordPress reads the header; the constant drives asset cache-busting, so browsers could be served stale CSS/JS.	Both aligned and released as 2.0.2 , reflecting a build containing security fixes.

Process defect — the underlying cause

Defects 1 and 2 share a root cause worth stating plainly: **the project maintained three divergent copies of the same code** — the development sandbox, the private repository, and the published open-source repository — with no synchronisation step and no verification that the published copy runs. Bug 2 is the clearest illustration: a single commit added both a function and its callers, but only the callers reached the public repository. Two of the three call sites happened to be `function_exists()`-guarded, which masked the failure enough for it to survive review.

The remediation is not only the code fix. It is \$3's CI, which loads the published build on every push, and the reconciliation recorded in \$7.

6. Automated Vulnerability Checks

The Acceptance Criterion names *"NodeZero or similar"*. We used **semgrep** — an industry-standard static application security testing tool — with its maintained PHP and security-audit rule packs, because it produces machine-verifiable output a reviewer can reproduce from the repository. Full analysis, including the manual assessment, is in the companion Security Audit Report.

```
$ semgrep scan --config=p/php --config=p/security-audit code/
Ran 122 rules on 73 files: 1 finding.
errors: 0
```

Finding	Location	Verdict
php.lang.security.injection.echoed-request — echoing user input risks XSS	includes/admin-nft-monitor.php:119	False positive. The value is cast to integer on the preceding line (<code>\$n = (int) \$_GET['uploaded'];</code>) before being echoed, so no injection is reachable. Triaged, not suppressed.

The scan runs in CI on every push, so this result is continuously re-verified rather than a point-in-time claim.

7. Repository Reconciliation

The three working trees were reconciled so that the published artifact matches what production runs. Shared modules are now byte-identical across all three.

Change	Direction
<code>valt_resolve_artist_id()</code> (Pods-safe artist resolution)	private → public, sandbox
HD song thumbnails (<code>medium</code> → <code>large</code>)	private → public, sandbox
Conditional module loading; XSS, CSRF and Stripe-guard fixes; version 2.0.2	applied to all trees

Two files previously reported as divergent (`discovery.php` , `seed-data.php`) proved to differ only in line endings (CRLF vs LF), not content. Line-ending conventions were preserved per-file so that the change history stays reviewable.

8. Known Limitations

Recorded here rather than omitted, because a report claiming no residual risk would not be credible.

- **The plugin depends on the theme.** `valt-platform` calls `valt_svg_*`() presentation helpers defined in `valt-theme` . `code/README.md` describes the two components as "intentionally decoupled"; in this direction that is not currently accurate. The pair always ships together so it is not a live defect, but activating the plugin under a different theme would fatal. A test pins the coupling to the known icon helpers so it cannot widen silently; decoupling is a roadmap item.
- **Feature-gated modules are unreachable, not absent.** The public build omits gamification, leaderboard and campaigns. Calls into them are safe only because `valt_feature_enabled()` defaults those flags to `false` . A test asserts those defaults hold, since flipping one would turn every such call into a fatal.
- **The suite tests loading and structure, not behaviour.** It does not exercise minting, wallet connection or gating against a live chain; those remain verified manually and on-chain, as in Milestone 2. Behavioural integration tests are a roadmap item.